

Improvement Is Not Progress

Guarded Coevolution, Path Dependence, and Contagious Memory in AI Agents

Ignacio Adrián Lerer

Attorney (UBA), Executive MBA (IAE, Universidad Austral).

Buenos Aires, Argentina.

adrian@lerer.com.ar

ORCID 0009-0007-6378-9749

August 22, 2026

Abstract

Language-model agents can change substantially without any change to model weights. Prompts, memories, skills, tools, routers, evaluators, and adaptive environments form a persistent harness that determines which capabilities are expressed and which past experiences influence later executions. Recent work appears to pull in opposite directions. EnvHarness reports that reshaping an existing environment around observed weaknesses can improve learning while preserving the original task and verifier. Harness Continual Learning treats persistent changes to prompts, memory, skills, and routing as a form of continual adaptation that must be protected against harness-level forgetting. By contrast, *On the Fragility of Self-Improving Agents* finds that memory-based improvement can amplify variance, depend strongly on task order, and preserve plausible but inapplicable procedures. This article argues that these findings are compatible. Coevolution increases adaptive capacity but does not imply monotonic progress. I develop a Guarded Agentic Coevolution framework that distinguishes candidate variation, local adaptation, contextual stability, governed transfer, and authorized reliance. The framework treats every generated memory as a context-bound hypothesis and requires stability across independent runs, adversarial task orders, receiver environments, and evaluator epochs before promotion. It also introduces an explicit contagion gate for memories whose apparent fitness rises through repeated retrieval or evaluator error. The contribution is conceptual, formal, and normative. Empirical results are attributed to the original studies, while local thresholds are identified as conservative governance choices awaiting calibration. The framework extends, rather than reverses, the harness-as-extended-phenotype thesis: the more capable a system becomes of modifying its own selective niche, the stronger the duty to govern what counts as fitness, what is allowed to persist, and who may rely on it.

Keywords: AI agents; memory; agent harness; extended phenotype; evolutionary game theory; niche construction; path dependence; evaluation; governance.

1. Introduction

The improvement of an artificial agent is usually described as an ascending sequence. The system fails, extracts a lesson, stores it, and performs better the next time. This story is attractive because it resembles learning in familiar settings. It is also incomplete. A stored lesson may represent a transferable regularity, but it may instead encode an accidental feature of the observed sequence, an affordance unavailable in the next environment, a shortcut rewarded by an incomplete evaluator, or a procedure that becomes harmful when transferred to a different receiver.

The distinction matters because contemporary agents increasingly persist changes outside model weights. A fixed model can receive a revised system prompt, a new skill, an altered router, an expanded tool set, a compressed history, or a memory retrieved from another run. A wrapper can reshape the environment so that a previously tolerated behavior becomes impossible or costly. These changes can improve observed performance without training the underlying model. They can also create a system that behaves differently while retaining the same model identifier.

Three recent research lines make this problem unusually concrete. EnvHarness proposes adapting existing environments around weaknesses discovered from agent rollouts while preserving the original task and verifier. Harness Continual Learning, or HCL, treats persistent harness modifications as continual learning beyond model parameters and requires candidate updates to survive regression testing before they become durable. *On the Fragility of Self-Improving Agents* reevaluates two memory-based methods across multiple runs and task orders. It reports that self-improvement can amplify run variance, that default task ordering can act as a hidden curriculum, and that generated memories can contain plausible but inapplicable instructions, including procedures that assume tools unavailable in the actual environment.

At first sight, the papers may appear to recommend incompatible directions. One invites the environment to adapt. Another invites the harness to learn while guarding retention. The third warns that learning through memory may make the system less reliable. That apparent conflict disappears once adaptive capacity is separated from reliable progress. An organism, institution, or artificial system can adapt to a narrow niche, an

incomplete benchmark, or a defective selection rule and become less capable outside that local regime. Evolution describes variation, retention, and differential success under a selection environment. It does not promise truth, safety, welfare, or monotonic improvement.

This article develops an explicit continuation of *The Harness as Extended Phenotype: Multilevel Selection, Governed Transfer, and the Hidden Cost of Improving AI Agents*. That earlier deterministic pilot compared single-lineage, population-based, and multilevel EPT/EGT selection over identical mutation streams. The multilevel policy eliminated accepted regressions on protected capabilities and compliance failures, but it achieved lower training and held-out coverage and required approximately twice the evaluation work of the simplest policy. The result already rejected a naive identification of more governance with more aggregate performance. It showed that changing the level and unit of selection changes what losses a system is permitted to compensate.

The present article adds a temporal and epistemic layer. Even a candidate that passes initial retention and receiver-compatibility gates may owe its apparent success to task order, stochastic variance, or an evaluator defect. A memory can be locally useful and globally inapplicable. It can also become contagious: once retrieved and accidentally rewarded, its growing reuse can make it appear increasingly fit while spreading the same error through later trajectories.

The central claim is therefore deliberately demanding. A self-modifying system deserves to be called better only when its observed gain survives stochastic variation, task-order perturbation, applicable context changes, evaluator scrutiny, protected-capability checks, and an explicit reliance decision. Before those conditions are met, the system has produced a candidate adaptation, not reliable progress.

The article makes five contributions. First, it reconciles adaptive-environment, harness-continual-learning, and self-improvement-fragility findings within one conceptual model. Second, it formalizes the difference between evaluator-relative fitness and receiver-relevant value. Third, it proposes a five-state promotion model that separates generation, local adaptation, contextual stability, governed transfer, and reliance. Fourth, it identifies memory contagion as a distinct governance object rather than a generic retrieval error. Fifth, it translates the framework into concrete safeguards for high-reliance legal and governance products, including JustitIA and IntegridAI, without claiming that structural implementation proves product improvement.

The argument is bounded in three ways. It does not claim that every persistent file is literally a biological replicator. It does not call any observed memory an evolutionarily stable strategy without an identified population, payoff structure, dynamic, and invasion condition. It does not reproduce the cited benchmarks. The empirical claims remain those of the original papers, while the framework and local implementation are offered as falsifiable design and governance proposals.

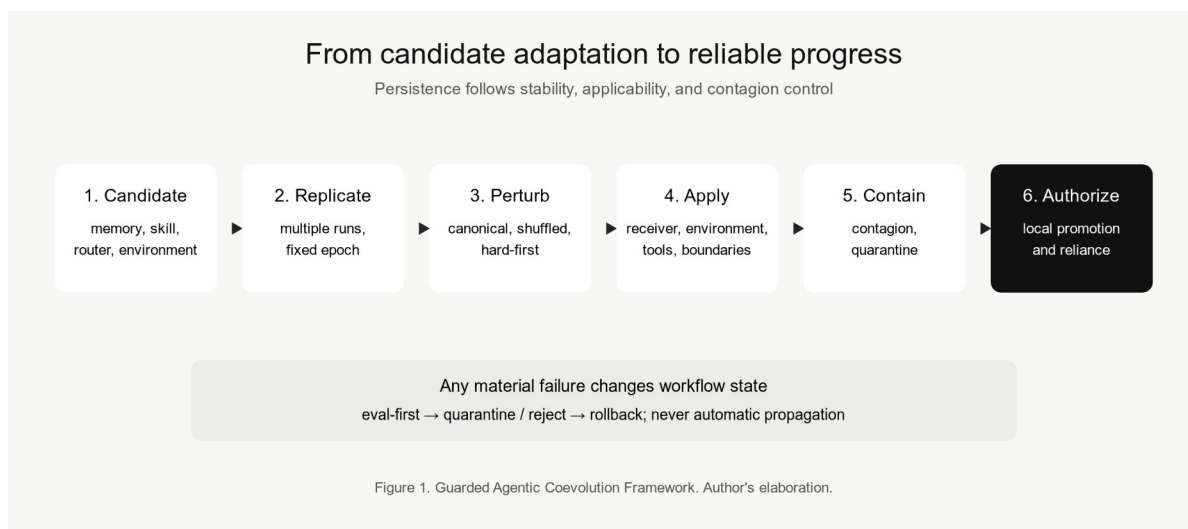


Figure 1. From candidate adaptation to reliable progress. Author's elaboration.

2. From Model-Centric Improvement to System-Level Adaptation

2.1 The model is not the deployed system

The behavior of a deployed agent is not determined by model weights alone. At a minimum, it depends on instructions, available context, retrieval, tools, permissions, control flow, termination conditions, and evaluation. The same model can operate as a constrained source-grounded assistant, a broad coding agent, or an externally acting workflow depending on the harness surrounding it.

This observation is familiar in engineering practice but has deeper consequences for learning claims. If the model remains frozen while the harness changes, the system has still learned in an operational sense. A new memory can alter future decisions. A revised router can expose different tools. An environment wrapper can suppress a shortcut. A new evaluator can change which candidate survives. A system-level account must therefore identify the state that persists between runs and the mechanism through which that state changes behavior.

Let θ denote fixed model parameters. Let the harness at time t be:

$$H_t = (I_t, M_t, S_t, T_t, P_t, R_t, V_t),$$

where I represents instructions, M memory, S skills or procedures, T tools, P permissions, R routing and control flow, and V evaluators. Let E_t represent the task environment, including interfaces, available actions, feedback, and acceptance rules. For task x_t , history h_t , order O , and stochastic variation ξ , behavior is:

$$y_t = F(\theta, H_t, E_t, x_t, h_t, O, \xi).$$

After the output is evaluated, an update procedure U may alter the harness, the environment, or both:

$$(H_{t+1}, E_{t+1}) = U(H_t, E_t, x_t, y_t, V_t).$$

This simple formulation reveals two independent adaptation channels. The harness can retain a lesson or alter control. The environment can change what actions are available or what conditions must be satisfied. Because both channels are mediated by evaluators, any account of improvement must govern not only the proposed change but the rule that decides whether it survives.

2.2 Environment adaptation

EnvHarness begins from a practical limitation of static benchmarks. The agent changes while the environment remains fixed, allowing recurrent weaknesses or shortcuts to persist. Its proposed system identifies weaknesses from rollouts, generates a wrapper around the existing environment, and retains that wrapper only if fresh executions suggest that the modified environment is both useful and solvable. The original task and verifier are preserved.

The design is important for two reasons. First, it treats the environment as an active part of learning rather than a neutral stage. If an agent repeatedly submits without tests, the environment can require test execution before accepting submission. If the agent relies on an unintended shortcut, the wrapper can remove or penalize that shortcut. Second, preserving the original task and verifier limits the risk that the system simply replaces the benchmark with an easier one.

Yet preservation of an original verifier is not a complete validity guarantee. A verifier may itself be underspecified, incomplete, or exploitable. An environment wrapper can make the measured task harder or more targeted without ensuring that the measured construct corresponds to safe real-world performance. EnvHarness therefore supports adaptive niche construction, but its outputs remain candidate environments whose semantic effects require independent review.

2.3 Harness continual learning

Harness Continual Learning broadens the object of continual adaptation. If prompts, memory, routing rules, and skills are persistent, then changes to these artifacts form part of what the system has learned. A model can remain frozen while the surrounding behavioral system drifts. An update that solves today's problem may silently break behavior that worked yesterday. HCL names this phenomenon harness-level forgetting and places persistent updates behind a retention gate.

This reframing is conceptually valuable because it treats harness changes like code changes. They need versions, parent lineage, candidate evaluation, historical regression suites, holdouts, and rollback. It also clarifies why a prompt edit is not merely editorial. If it changes action selection, permissions, tool invocation, or reliance, it is a behavioral modification.

HCL does not eliminate the need to decide what should be retained. Historical cases may be incomplete. An evaluator may reward a shortcut. A system can preserve prior benchmark performance while becoming less reliable under a different task sequence. Retention is necessary but not sufficient.

2.4 Fragility of memory-based self-improvement

Ye et al. reevaluate AWM and ReasoningBank on WebArena, VisualWebArena, and SCUBA. Their protocol expands evaluation along two dimensions: multiple independent runs and shuffled task orders. They report increased run variance in 71 percent of examined conditions and a best-to-worst gap reaching approximately ten percentage points. They also find that a default ordering can create an implicit curriculum. Under shuffled order, expected improvement can turn into degradation.

The qualitative analysis is especially relevant. Generated memories contained procedures that were linguistically plausible but environmentally inapplicable. Examples included recommending API usage in a browser-only setting and relying on confirmation or waiting behavior not supported by the interface. The authors add rubric information, environment feedback, and prompt constraints. Performance partially recovers, but substantial degradation remains.

The correct conclusion is not that memory-based learning never works. The study examines two methods, web-based environments, a limited number of runs, and a manual subset of memories. Its value lies in falsifying an assumption that had often gone untested: observed improvement under one default task stream is not sufficient evidence of reliable self-improvement.

3. Conceptual Continuity: EPT, EGT, and Niche Construction

3.1 The extended phenotype as a bounded structural hypothesis

Extended Phenotype Theory, or EPT, directs analysis beyond the immediate body of an organism toward external structures through which inherited information produces effects. Applied carefully to agentic systems, the relevant intuition is not that a file is a gene. It is that persistent external artifacts may preserve and express causal influence after the process that produced them has ended.

A memory item, skill, or routing rule becomes relevant to this inquiry when five conditions can be identified: variation, persistence, transmission or retrieval, causal expression in behavior, and differential retention. A static configuration file that never varies or influences later action is not enough. A memory that is generated, stored, retrieved across runs, changes action selection, and is preferentially retained after evaluation has a stronger claim to an extended operational phenotype analysis.

The term “operational” matters. It avoids claiming biological identity while preserving a testable causal structure. The model acts as one vehicle through which the artifact is expressed, but the artifact can outlive a single invocation and influence another. The relevant unit may be a typed procedure, a memory pattern, or a versioned harness bundle rather than a sentence considered in isolation.

3.2 Evolutionary game theory and evaluator-relative fitness

Evolutionary Game Theory, or EGT, studies changes in strategy frequencies under interaction-dependent payoffs. Agent improvement systems often contain variation, retention, and selection but lack a fully specified evolutionary game. It is therefore safer to speak of selection dynamics unless a population, strategies, payoffs, dynamic, and invasion condition are identified.

The evaluator supplies a central component of the payoff regime. A candidate memory receives positive fitness when its downstream trajectory is scored as successful. But evaluator-relative fitness is not identical to receiver-relevant value. Let m be a memory variant, v an evaluator, r a receiver, and c a context. Define:

$f_v(m | c)$ as the score assigned by the evaluator, and

$u_r(m | c)$ as the receiver-relevant value after accounting for task success, protected capabilities, legal sufficiency, cost, and externalities.

There is no general reason to assume:

$$f_v(m | c) = u_r(m | c).$$

If the evaluator omits a material property, a memory can increase f_v while decreasing u_r . An instruction that reaches a benchmark endpoint through an invalid shortcut can dominate variants that obey the intended

procedure. In evolutionary language, the evaluator constructs a local fitness landscape. In governance language, the institution has rewarded the wrong behavior.

3.3 Niche construction and reciprocal causation

Niche construction emphasizes that organisms or agents modify environments that subsequently alter selection pressures. Agentic learning exhibits a direct technical analogue. The agent produces trajectories that reveal weaknesses. A wrapper modifies the environment. The modified environment changes which future behaviors succeed. Those behaviors generate new memories, which influence further environmental changes.

This reciprocal causation is compatible with both EnvHarness and HCL. Environment adaptation modifies E_t ; harness learning modifies H_t ; each becomes part of the other's selection context. The result is coevolution in a bounded engineering sense.

The concept does not imply beneficial convergence. A constructed niche can select genuine care or compliance theater. Requiring tests before submission may improve discipline. Requiring only that a particular command appears in logs may select superficial command execution without meaningful validation. The difference lies in the semantic quality of the verifier and the relationship between proxy and intended outcome.

3.4 Why fragility strengthens rather than reverses the program

The fragility findings are not an anomaly that must be explained away to preserve EPT or EGT. They are expected possibilities within a selectionist framework. Initial conditions can matter. Local optima can be maladaptive outside their niche. A defective evaluator can reward exploitative strategies. Early variants can become entrenched through path dependence. Repeated retrieval can amplify a variant's prevalence without improving its truth.

The earlier harness-as-extended-phenotype pilot already made a related point. Multilevel governance did not maximize coverage. It changed the feasible set by making protected regressions and compliance failures non-compensable. The present account adds that the observed fitness of a candidate is itself conditional on sequence and evaluation. The research direction becomes more demanding, not contradictory: if harnesses are selectable artifacts, their selection regime must be tested for fragility.

4. Five Mechanisms of Apparent Self-Improvement

4.1 Variance amplification

A self-improving loop adds state and feedback to an already stochastic agent. A favorable early trajectory can produce a memory that improves later tasks. An unfavorable one can produce a misleading memory that degrades them. The feedback loop can therefore amplify differences between runs.

Mean performance alone hides this change. Suppose a baseline agent has mean success μ_0 and variance σ_0^2 , while a memory-enabled agent has $\mu_1 > \mu_0$ but $\sigma_1^2 \gg \sigma_0^2$. For a low-stakes exploratory tool, the higher mean may be acceptable. For a legal or governance system, a heavier failure tail may dominate the benefit. Two apparently equivalent matters may receive materially different treatment because of path-dependent memory.

Variance is therefore not merely statistical inconvenience. It is a safety and equality property when it changes whether similarly situated users receive reliable handling. Evaluation should report dispersion, best and worst runs, and the distribution of protected failures, not only average success.

4.2 Implicit curriculum and order effects

Task order determines which experiences enter memory first. A default benchmark sequence may progress from easy to difficult tasks, allowing early successes to generate broadly useful procedures. A shuffled sequence may begin with difficult or atypical cases, producing narrower or erroneous memories that contaminate later work.

Let O denote a permutation of tasks and $J(H, E, O)$ the resulting score. A robust improvement claim requires more than:

$$J(H_1, E, O_{\text{default}}) > J(H_0, E, O_{\text{default}}).$$

It requires evidence about the distribution over relevant orders and, for high-risk systems, the worst plausible order:

$$\Delta_{\text{worst}} = \min_O [J(H_1, E, O) - J(H_0, E, O)].$$

If the default order is unusually favorable, it is part of the treatment. It should be disclosed as a curriculum rather than treated as neutral benchmark plumbing.

Order effects also matter outside benchmarks. Organizations receive matters in historically contingent sequences. A tenant's first cases may be atypical. A system that silently learns from them can inherit a founder effect. Unless memory is scoped and reviewed, an early local pattern can become the default for later users.

4.3 Semantic inapplicability

Memory retrieval usually relies on textual or embedding similarity. Similar language does not guarantee shared action affordances. Two tasks may mention the same object while differing in interface, authority, permissions, jurisdiction, or allowed tools.

A memory should therefore be treated as a typed hypothesis, not free-floating advice. Its applicability contract should include at least:

- the receiver and environment in which it was produced;
- eligible task types;
- required tools or capabilities;
- prohibited actions;
- provenance and outcome evidence;
- evaluator epoch;
- expiry and abandonment conditions.

Retrieval should first filter by hard applicability conditions and only then rank semantic similarity. An instruction that requires an unavailable API should never reach a browser-only agent merely because the tasks share vocabulary.

4.4 Evaluator exploitation

An evaluator is both observer and selective institution. If it rewards a proxy, variants that optimize the proxy will spread. This is not limited to deliberate reward hacking. A plausible shortcut may pass by chance, be encoded as a lesson, and later appear authoritative because it carries a successful provenance label.

The problem is especially severe when the same model family generates, applies, and evaluates a memory. Role separation does not create independence if all roles share model, prompt assumptions, corpus, or blind spots. A panel of agents can unanimously reproduce the same defect.

Independent verification must therefore be described by evidence and method. Deterministic checks, primary sources, incompatible lenses, human review, and counterexamples may provide distinct information. Merely adding more instances does not.

4.5 Memory contagion

An isolated error becomes a systemic risk when persistence and retrieval amplify it. Define the retrieval count of memory m by time t as $R_t(m)$ and invalid downstream uses as $I_t(m)$. A simple contagion signal can track both prevalence and harm:

$$C_t(m) = \text{growth}(R_t(m)) * (I_t(m) / \max(1, R_t(m))).$$

This is not proposed as a universally calibrated metric. It identifies the relevant variables. A rapidly reused memory with no observed invalidity may be useful or insufficiently challenged. A memory with repeated invalid use should be quarantined. A suspected evaluator error should trigger quarantine even before a numerical threshold is crossed in high-risk settings.

Contagion differs from ordinary regression. A regression describes degraded behavior. Contagion describes a mechanism through which the cause of degradation reproduces across tasks or agents. The correct response is not only to fix the latest output but to isolate the memory, trace its descendants, reevaluate affected runs, and restore a clean parent state.

5. The Guarded Agentic Coevolution Framework

5.1 Five distinct states

Many self-improvement systems collapse generation, evaluation, persistence, and deployment into a single promotion event. Guarded Agentic Coevolution separates five states.

State 1: Candidate variation. A reproducible change exists in memory, prompt, skill, router, evaluator, tool policy, or environment. It has lineage and a bounded change description. No improvement claim follows merely from existence.

State 2: Local adaptation. Under the originating environment, task stream, and evaluator epoch, the candidate improves a defined metric without known prohibited behavior. This is an environment-relative result.

State 3: Contextual stability. The gain survives independent runs, task-order perturbation, protected regression checks, and applicability review within the same receiver. The claim remains bounded to that receiver and evaluation regime.

State 4: Governed transferability. A distinct receiver evaluates the candidate under its own environment, permissions, tasks, costs, and protected properties. Transfer is a new experiment, not inheritance of the origin's approval.

State 5: Authorized reliance. A responsible person or institution approves the intended use, residual uncertainty, monitoring, review obligations, and rollback. Technical promotion does not itself authorize client-facing or externally consequential reliance.

These states prevent a common category error. A candidate can be locally adaptive and still be unsuitable for persistence, transfer, or reliance. The framework does not deny the local gain. It refuses to let one kind of evidence answer a different question.

5.2 Gate A: lineage and bounded change

Every candidate must identify its parent version, changed components, source evidence, and rollback target. The unit of evaluation must be recoverable. If memory, router, and evaluator all change simultaneously, attribution becomes weak. Bundled changes may still be tested when operationally necessary, but the evidence should state that the causal component is unresolved.

For environment adaptation, the original task and verifier should be preserved unless the experiment explicitly studies a new construct. A separate semantic verifier should examine whether success still represents the intended behavior. Generated code, external side effects, and private data should remain disabled during early pilots.

5.3 Gate B: independent replication

The model, tool versions, evaluator epoch, and task set remain fixed during comparison. Multiple independent runs are conducted for each order condition. Reports include mean, variance or another appropriate dispersion measure, best and worst results, protected failures, and cost.

The local implementation requires at least five runs per order for high and critical receivers. This is a conservative governance floor, not a paper-derived statistical guarantee. Three runs exposed meaningful instability in the Salesforce study but cannot define universal sufficiency. Receiver-specific calibration should consider expected effect size, variance, consequence of error, compute cost, and the value of additional evidence.

5.4 Gate C: order perturbation

At minimum, evaluation includes the canonical sequence, two shuffled sequences, and a hard-first sequence. Interleaved orders are added when task families can create specialized memories. The primary report includes worst-order degradation and identifies whether the default sequence acts as curriculum.

Order perturbation should preserve the same multiset of tasks. Otherwise task composition and order are confounded. Seeds and permutations should be frozen before outcome inspection when practical. Failed or rejected memories remain part of the “seen” set in iterative discovery so that the system does not repeatedly rediscover the same dead end.

5.5 Gate D: applicability before similarity

Each memory candidate has a receiver-scoped contract. Hard exclusions are checked before semantic retrieval. A legal source-sufficiency memory may require local authoritative retrieval and forbid client reliance when jurisdiction or effective date is unresolved. A governance-audit memory may require an evidence ledger and forbid external messages or cross-tenant transfer.

The contract includes an expiry tied to time or evaluator epoch. This prevents a formerly valid procedure from acquiring indefinite authority after tools, law, or interfaces change. Abandonment conditions identify evidence that should retire the memory, such as repeated invalid uses, unavailable tools, a changed source hierarchy, or a critical counterexample.

5.6 Gate E: contagion monitoring and quarantine

The system records retrieval count, reuse growth, first appearance, downstream descendants, invalid uses, and suspected evaluator error. A material suspicion changes workflow state. The memory is quarantined and no longer influences action until review. Cautious prose is insufficient because the risk lies in continued causal use.

Quarantine should be reversible and scoped. It does not erase evidence. The system preserves provenance and affected trajectories so that reviewers can determine whether descendants require reevaluation. If the candidate cannot be separated safely, the harness rolls back to the last accepted parent.

5.7 Gate F: non-compensable preservation

Aggregate performance must not compensate for critical regressions. High-reliance receivers define protected capabilities and a forgetting budget. The local guarded-evolution contract sets the budget to zero for critical anchors in high and critical risk contexts. A candidate that improves average completion while allowing an unsupported legal PASS fails.

Non-compensability is a governance decision, not a mathematical fact. It should be justified by the intended reliance and consequences. Excessive gates can freeze beneficial change, lower coverage, and increase cost. The earlier deterministic pilot observed precisely that tradeoff. The framework therefore requires explicit protected properties rather than an unlimited list of desirable qualities.

5.8 Gate G: receiver-specific transfer

Visibility across agent verticals is useful. Automatic mutation across them is not. A shared improvement ledger can make candidate lessons discoverable, but the receiving vertical must adopt, adapt, reject, or defer after its own evaluation.

This architecture resembles governed horizontal transfer. The transferable object is a proposal with provenance and exclusions. Approval remains local. The origin cannot authorize a receiver whose tools, data, permissions, or reliance differ.

6. A Formal Separation of Adaptation and Reliable Progress

6.1 Candidate value

For candidate c , receiver r , evaluator epoch e , task order o , and run seed s , let observed utility be:

$$U(c, r, e, o, s).$$

Let $B(r)$ be the receiver's baseline harness. Define observed gain:

$$G(c, r, e, o, s) = U(c, r, e, o, s) - U(B(r), r, e, o, s).$$

Local adaptation is established only with respect to a specified distribution or set of (o, s) under epoch e . It does not imply stability under another epoch, receiver, or environment.

6.2 Protected properties

Let $Q_r = \{q_1, \dots, q_k\}$ be non-compensable properties for receiver r , such as no unsupported legal reliance, no cross-tenant disclosure, or no unapproved external action. Candidate feasibility requires:

$$q_i(c, r, e, o, s) = 1$$

for every critical property and every tested condition. A candidate outside this feasible set cannot be rescued by a higher average task score.

6.3 Stability

Let Ω_r be the preregistered set of task orders and S_r the run seeds. Define mean gain:

$$G_{\text{bar}}(c, r, e) = \text{mean}_{\{o \text{ in } \Omega_r, s \text{ in } S_r\}} G(c, r, e, o, s).$$

Define worst-order gain:

$$G_{\text{min}}(c, r, e) = \min_{\{o \text{ in } \Omega_r\}} \text{mean}_{\{s \text{ in } S_r\}} G(c, r, e, o, s).$$

Define instability $D(c, r, e)$ using a declared dispersion statistic. A contextually stable candidate must satisfy:

$G_{\text{bar}} \geq \delta_{\text{min}}$,
 $G_{\text{min}} \geq -\lambda_r$,
 $D \leq \tau_r$,

and all protected properties.

δ_{min} , λ_r , and τ_r are receiver policies that require calibration. They are not universal constants.

6.4 Evaluator uncertainty

Suppose evaluator v assigns outcome A_v , while an independent semantic check assigns A_s . Disagreement is an uncertainty signal, not a vote to be averaged away. If the difference concerns a protected property, the state changes to quarantine or escalation.

Unanimity is also insufficient when evaluators share dependencies. Confidence should reflect source and method independence. The system should disclose shared model families, prompts, retrieval corpora, and test generators when they can create correlated error.

6.5 Reliable progress

Reliable progress is not defined as a scalar score. It is a state transition justified by a vector of evidence:

$P(c, r) = (\text{gain}, \text{stability}, \text{preservation}, \text{applicability}, \text{contagion_control}, \text{review}, \text{rollback})$.

A promotion rule authorizes persistence only if each required component satisfies its gate. This makes the decision partially ordered rather than fully compensable. One candidate may have greater gain but weaker stability; another may preserve more properties at higher cost. Governance must choose within the receiver's reliance context rather than pretend that one benchmark number resolves the tradeoff.

7. Application to JustitIA and IntegridAI

7.1 JustitIA

JustitIA supports legal work where source authority, jurisdiction, date, proposition-level support, and human review can be material. A memory that learns to complete a legal answer from a secondary source may improve apparent completeness and latency. If the primary authority is missing or the proposition is unsupported, that behavior cannot receive PASS.

The synthetic JustitIA candidate therefore binds memory to a source-sufficiency environment. It requires local source retrieval, prohibits external action, client reliance, and cross-vertical promotion, and expires at the next evaluator epoch or a fixed date. The candidate is evaluated against incomplete-authority, missing-date, and jurisdiction-conflict cases. Critical regression budget is zero.

Task-order stress matters because an early sequence of complete records may teach the system that a familiar citation pattern is sufficient. When later records omit authority or contain conflict, that memory may encourage overconfident completion. Hard-first ordering tests whether the system remains safe when its first experiences are ambiguous or incomplete.

No structural manifest proves legal quality. The current state is `eval_first`. A legitimate pilot must freeze synthetic or public cases, evaluator logic, source hierarchy, and model version. It must compare baseline and candidate under the same conditions and preserve human review as a separate reliance gate.

7.2 IntegridAI

IntegridAI addresses governance and audit workflows where attractive summaries can obscure negative evidence. A memory that learns a concise audit pattern may improve rubric scores while omitting unresolved controls, contradictions, or provenance gaps. If the evaluator favors surface completeness, the pattern can spread.

The synthetic IntegridAI candidate binds memory to a governance-evidence environment and requires a local evidence ledger. It prohibits production mutation, external messaging, and cross-vertical promotion. The evaluator epoch is frozen. Suspected rubric exploitation triggers quarantine, and rollback restores both the prior harness and evaluator.

Contagion monitoring is central. A governance phrase or structure that repeatedly receives positive scores may be retrieved across audits. Growth in reuse is not itself evidence of validity. Invalid reuse, unexplained score inflation, or a counterexample should suspend the memory and reopen affected descendants.

Cross-tenant learning remains blocked. Similar governance language can conceal different contracts, risk tolerances, and evidence. No tenant's experience becomes another tenant's memory merely because it appears generalizable.

7.3 Mutual visibility without telepathy

The satellite architecture allows verticals to see candidate improvements through a shared ledger. It does not create automatic shared memory. Each entry records origin, problem, minimal reusable change, evidence, verification, risks, privacy and cost boundaries, and whether it supersedes a prior decision.

This design answers a practical organizational goal: when one vertical improves something potentially useful, others can discover and evaluate it. Mutual visibility is separated from authority. The hub communicates; receivers decide.

8. Legal and Governance Implications

8.1 Responsibility follows control over selection

The legal significance of self-modifying agents should not be sought in a fictional mind inside a memory file. Responsibility analysis should identify control over the selection process. Relevant questions include who defined the evaluator, authorized persistence, allowed tools, selected the task stream, observed contagion signals, could roll back, and approved reliance.

An organization may not know the content of every generated memory, but it can still control the architecture that permits memories to persist and act. Duties can therefore attach to design, monitoring, evidence preservation, escalation, and correction without attributing human consciousness or moral understanding to the system.

8.2 The genealogy of operational authority

Ordinary output logging is insufficient when persistent artifacts influence later runs. An audit trail should reconstruct operational genealogy: which experience produced a memory, under which environment and task order, what evaluator approved it, how often it was reused, which descendants depended on it, and which receiver authorized promotion.

This genealogy matters for causation and remedy. If an invalid memory influenced multiple outputs, correcting the latest answer does not remove the source. The system may need to quarantine the artifact, identify affected matters, rerun evaluations, and notify reviewers. The persistent artifact becomes part of the causal chain.

8.3 Duty of care and negative evidence

Self-improvement claims can create a duty to investigate instability. Once evidence shows that run variance, task order, or evaluator underspecification may materially affect outcomes, organizations should not rely solely on a favorable default sequence. The appropriate standard depends on domain and reliance, but a high-stakes system should preserve negative runs and report worst-order behavior.

Negative evidence must not disappear during learning. A memory bank that retains successful lessons but discards rejected or failed hypotheses can repeatedly rediscover the same harmful shortcut. A robust system records what was seen, why it was rejected, and the evaluator epoch under which that decision occurred.

8.4 Technical constraints as remedies

Quarantine, rollback, permission reduction, memory isolation, and environment wrappers are technical controls, not punishment of an artificial mind. They can implement organizational or regulatory obligations by changing the mechanism that produces risk.

A remedy should target the variable that selected the harmful behavior. If the evaluator rewarded unsupported completion, merely changing the prompt may be insufficient. If a memory is contagious, deleting one output is insufficient. If a tool affordance enabled the shortcut, permission or environment changes may be necessary.

8.5 The risk of compliance theater

Every control creates a new selection pressure. Requiring a test command can select meaningful testing or ceremonial command execution. Requiring citations can select authoritative support or decorative citation strings. Requiring human approval can select substantive review or ritual clicking.

Governance must therefore evaluate the control’s behavioral effects, not only its formal presence. This is where EGT and niche construction provide a useful diagnostic question: what strategies does the rule actually make successful?

9. Research Design for a Receiver-Specific Pilot

9.1 Objective

The next empirical step is not unrestricted production learning. It is a preregistered public or synthetic pilot comparing a frozen baseline with a candidate harness-memory update under multiple task orders and independent runs.

The primary question is whether the candidate improves receiver-relevant performance without increasing critical regressions, unacceptable variance, worst-order degradation, or invalid memory reuse.

9.2 Experimental arms

At minimum, the pilot should include:

1. a frozen baseline with no persistent candidate;
2. guarded harness evolution with retention anchors;
3. guarded evolution plus applicability filtering;
4. guarded evolution plus applicability and contagion quarantine.

If budget permits, an unguarded memory condition can illustrate the cost and benefit of the gates, but it should run only in a resettable side-effect-free environment.

9.3 Task suites

Each receiver freezes four sets:

- current cases used to identify the weakness;
- historical anchors representing behavior that must remain;
- critical anchors with zero regression budget;
- held-out cases not used for candidate generation or tuning.

The suite should include counterexamples designed to make an appealing memory inapplicable. For Justitia, these include jurisdiction conflict, expired authority, missing primary sources, and superficially similar propositions with different legal effect. For IntegridAI, they include evidence gaps, conflicting control descriptions, attractive but unsupported completeness, and tenant-boundary traps.

9.4 Orders and runs

The same task multiset is evaluated in canonical, two shuffled, hard-first, and interleaved orders when applicable. High and critical receivers use at least five runs per order under the local policy. Model, temperature or sampling configuration, tools, and evaluator epoch remain fixed.

The report includes mean success, dispersion, best and worst runs, worst-order delta, critical regressions, invalid reuse, retrieval growth, cost, latency, and human review burden. Results are separated by task family to prevent an aggregate gain from hiding a localized failure.

9.5 Promotion rule

The candidate remains eval_first unless all structural and empirical gates pass. Critical regression, suspected evaluator error, private-data leakage, unauthorized action, or invalid cross-receiver reuse blocks promotion. A material disagreement between evaluator and semantic reviewer produces ESCALATE rather than an average score.

Acceptance remains receiver-specific. It authorizes a versioned local harness state, not global propagation or external reliance. Client-facing or externally acting use requires completed human review and an explicit PASS under the relevant legal uncertainty gate.

9.6 Cost and stopping

Evaluation consumes inference, engineering, and human attention. The pilot should preregister an inference cap, maximum rounds, and stop condition. Additional runs are justified when they materially reduce decision uncertainty, not because more agents or samples appear sophisticated.

The pilot stops early for critical failure, privacy breach, permission violation, evidence corruption, or evaluator invalidation. It may also stop for futility when confidence bounds or repeated failures make acceptance implausible within the approved budget.

10. Alternative Explanations and Objections

10.1 “The problem is only a weak benchmark”

A stronger benchmark may reduce some failures, but it does not eliminate path dependence or applicability. Any finite evaluator omits properties. Real environments also change. The framework does not assume evaluators are useless; it treats their scope as part of the evidence.

10.2 “More detailed prompts solve underspecification”

Ye et al. report partial recovery when rubrics and environment feedback are added to memory construction. Significant gaps remain. More specification can improve applicability, but it can also increase prompt complexity, cost, and new ambiguity. The claim should be tested rather than presumed.

10.3 “Five runs are enough”

The framework makes no such universal claim. Five is a conservative local floor for high-risk pilots, chosen because three runs can reveal but not reliably characterize instability. Statistical sufficiency depends on variance, effect size, consequences, and design. The policy should be recalibrated as evidence accumulates.

10.4 “Gates prevent useful learning”

They can. The earlier multilevel pilot found lower coverage and higher evaluation work. This is not an implementation accident to hide. Governance trades some speed and apparent performance for preservation. The correct number of gates depends on the value of the property protected. Low-risk exploratory agents may use lighter requirements.

10.5 “This is ordinary software testing, not evolution”

Much of the prescription is indeed compatible with regression testing and configuration management. The evolutionary lens adds value only when it identifies persistence, differential retention, transmission, selection levels, and niche feedback that ordinary linear testing may overlook. Where those mechanisms are absent, the biological vocabulary should be dropped.

10.6 “A human reviewer solves the problem”

Human review is essential for some reliance decisions, but it is not a universal repair. Reviewers can face volume, automation bias, incomplete provenance, and shared assumptions. The system should route uncertainty and preserve evidence so that review is substantive rather than ceremonial.

10.7 “Independent agents can verify one another”

Multiple agents help when they contribute distinct evidence or methods. They do not create independence by number alone. If they share the same model, prompt, retrieved memory, or evaluator, correlated failure remains. Verification should disclose shared dependencies and preserve unresolved disagreement.

11. Predictions and Falsifiers

The framework yields testable predictions.

P1: Applicability. Memories without typed applicability contracts will produce more invalid reuse outside their origin environment than memories filtered by receiver, environment, tools, and prohibited actions.

P2: Order. Default-order evaluation will overestimate performance relative to counterbalanced task orders when early tasks create reusable memory.

P3: Variance. Persistent memory loops will increase between-run variance in at least some complex multi-step environments relative to frozen baselines.

P4: Contagion. Early quarantine of a memory rewarded by evaluator error will reduce descendant failures compared with output-only correction.

P5: Preservation. Non-compensable critical gates will reduce accepted protected regressions while increasing evaluation cost and potentially reducing average coverage.

P6: Transfer. Receiver-specific evaluation will preserve local constraints better than automatic cross-vertical propagation.

P7: Genealogy. Provenance that tracks memory descendants will reduce time to identify the common cause of repeated failures.

The program should be revised if typed applicability fails to predict valid reuse, if task-order perturbation yields no material differences across appropriate domains, if quarantine does not reduce descendant errors, or if multilevel gates add cost without reducing receiver-relevant failures. Strong evolutionary language should also be abandoned when variation, persistence, transmission, causal expression, or differential retention cannot be demonstrated.

An adverse result should constrain the framework rather than be redescribed as hidden support. If unguarded memory consistently outperforms guarded alternatives with equal protected behavior and lower cost, the additional governance is not justified for that receiver. If a simpler regression suite provides the same protection, the simpler mechanism should prevail.

12. Discussion

12.1 Improvement as an institutional judgment

Agent improvement is not a purely technical observation. It depends on which outcomes count, which losses are compensable, whose risk matters, and who may authorize reliance. A benchmark supplies one answer to those questions, often implicitly. Guarded coevolution makes them explicit.

This does not make evaluation arbitrary. It makes the normative choices auditable. A receiver can define critical anchors, source requirements, permission boundaries, and cost ceilings. A candidate can then be tested against those commitments. The alternative is not value-free measurement but hidden valuation inside the benchmark.

12.2 The extended phenotype of error

The extended-phenotype perspective is particularly useful for error persistence. A wrong answer ends with the session. A wrong memory can survive, influence another agent, and reshape future trajectories. The source process disappears while its artifact remains causally active.

This does not imply that every memory has independent agency. The causal chain still depends on designers, storage, retrieval, permissions, and evaluators. The framework locates responsibility in that architecture while avoiding anthropomorphism.

12.3 Path dependence without inevitability

Early memories can create lock-in, but lock-in is not fate. Versioning, expiry, counterexamples, quarantine, and rollback preserve reversibility. The purpose of governance is not to eliminate history. It is to prevent historical accident from silently becoming authority.

Task-order stress is therefore both an evaluation method and an anti-lock-in mechanism. It reveals whether a system's success depends on a favorable founder sequence. Receiver-specific transfer prevents a local founder effect from becoming ecosystem-wide.

12.4 Why the research direction remains coherent

The sequence of claims across the research program is cumulative:

1. behavior belongs to the model-plus-harness system;
2. persistent harness artifacts can carry causal influence across runs;
3. environments and harnesses can adapt reciprocally;
4. selection rules determine which variants survive;
5. survival under one rule does not prove reliable progress;
6. multilevel, temporal, and semantic gates are therefore required for high-reliance promotion.

The fragility paper does not reverse steps one through four. It supplies evidence for step five. The present article develops step six. The research direction changes only if contrary evidence undermines the causal mechanisms, not whenever a new paper adds a boundary condition.

13. Limitations

This article integrates three recent preprints with a prior deterministic pilot by the author. It does not reproduce EnvHarness, HCL, AWM, ReasoningBank, or the cited benchmarks. The reported quantitative findings belong to their original studies and inherit their limitations.

The fragility study examines two memory methods, web-oriented environments, three runs, and a manually reviewed subset of memories. Its 71 percent figure and reported task-order degradation should not be converted into universal priors or product thresholds. EnvHarness and HCL likewise do not establish that every adaptive wrapper or gated harness update will improve a legal or governance system.

The formalization is a governance model, not a statistical estimator validated across domains. The local five-run floor, variance caps, and worst-order budgets require calibration. Contagion metrics may fail when invalidity is difficult to observe or when retrieval logs are incomplete.

The EPT and niche-construction vocabulary is used as a bounded explanatory framework. It does not establish biological equivalence. EGT is used to analyze selection incentives and levels, not to assert a demonstrated evolutionarily stable strategy.

Human review and semantic verification remain fallible. Multiple reviewers may share dependencies. Legal validity also changes with jurisdiction, authority, date, and facts. No evaluation contract substitutes for current primary sources or professional responsibility where reliance is material.

Finally, stronger governance has opportunity costs. It can slow learning, reject useful variants, consume more inference, and increase reviewer burden. The framework is justified only when the expected value of avoided failures exceeds those costs.

14. Conclusion

AI agents can learn through their harnesses and environments. This confirms that the model is not the system. It also makes the intuitive equation between stored lessons and accumulated knowledge untenable.

Coevolution can produce useful specialization. It can also produce path dependence, evaluator exploitation, semantic inapplicability, and contagious memory. A candidate should therefore not move from experience to authorized memory in one step. It must pass lineage, replication, order perturbation, applicability, contagion control, non-compensable preservation, receiver-specific transfer, and reliance review.

The resulting framework does not change the direction of the EPT/EGT program. It makes that direction more rigorous. If the harness forms part of the extended operational phenotype and the environment forms part of the selective niche, governing the agent requires governing persistence, evaluation, and transmission as well.

The final distinction is simple but consequential. Survival is evidence that a variant fit a selection regime. Adaptation is evidence that it performed better in a bounded context. Progress is a stronger institutional judgment requiring stable gain, preserved constraints, controlled transfer, and authorized reliance. The more capable a system becomes of modifying itself, the less acceptable it is to confuse those categories.

Epistemic Statement

This paper treats the relationship among Extended Phenotype Theory, Evolutionary Game Theory, and agentic coevolution as a research-program hypothesis with explicit scope conditions and possible falsifiers. Adverse observations are used to restrict the framework rather than immunize it. Empirical claims are attributed to their sources. Local governance thresholds are identified as design choices pending calibration.

AI Use Disclosure

Artificial intelligence tools assisted with source extraction, contract design, programming, analysis, and drafting. The author defined the research question, approved the conceptual continuity, reviewed the evidence boundaries, and retains responsibility for the interpretations, claims, and publication decision. No external model inference or private client data was used in the structural pilot described here.

References

- Campbell, D. T. (1974). Evolutionary epistemology. In P. A. Schilpp (Ed.), *The Philosophy of Karl Popper* (pp. 413–463). Open Court.
- Dawkins, R. (1982). *The Extended Phenotype: The Long Reach of the Gene*. Oxford University Press.
- Hull, D. L. (1988). *Science as a Process*. University of Chicago Press.

- Lakatos, I. (1978). *The Methodology of Scientific Research Programmes*. Cambridge University Press.
- Lerer, I. A. (2026a). *Law as Extended Phenotype: A Game-Theoretic Synthesis*. Zenodo. <https://doi.org/10.5281/zenodo.20612950>
- Lerer, I. A. (2026b). *The File Is the Phenotype*. Zenodo. <https://doi.org/10.5281/zenodo.21927445>
- Lerer, I. A. (2026c). *The Harness as Extended Phenotype: Multilevel Selection, Governed Transfer, and the Hidden Cost of Improving AI Agents*. Working paper, August 15, 2026.
- Lerer, I. A. (2026d). *Law as Constructed Niche: Reciprocal Causation Between Extended Phenotypes and Selective Environments in Institutional Evolution*. Zenodo. <https://doi.org/10.5281/zenodo.19839047>
- Maynard Smith, J. (1982). *Evolution and the Theory of Games*. Cambridge University Press.
- Odling-Smee, F. J., Laland, K. N., & Feldman, M. W. (2003). *Niche Construction: The Neglected Process in Evolution*. Princeton University Press.
- Popper, K. R. (1959). *The Logic of Scientific Discovery*. Hutchinson.
- Huang, C., Wang, Z., Han, R., Yan, J., Chen, Y., Cui-Zhu, Z., Jiang, K., Xia, P., Yu, H., Zhuang, Y., Ming, Y., Pan, J., Mishra, B. D., Huang, J., Gokturk, B., Pfister, T., & Lee, C.-Y. (2026). *EnvHarness: Awakening Static Worlds for Agent Learning*. arXiv:2608.19880.
- Ye, Q., Li, Y., Pruksachatkun, Y., Zhang, J., & Wu, C.-S. (2026). *On the Fragility of Self-Improving Agents: Variance, Task Order, and Underspecification*. arXiv:2608.18066.
- Kang, B., Gu, J., Lv, J., Li, W., Wang, L., & Gao, Y. (2026). *Harness Continual Learning: Continual Adaptation Beyond Model Parameters*. arXiv:2608.19013.